《算法分析与设计》

实验报告

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

实验预习（40%）	上机实验（40%）	文档排版（20%）	总分	批阅人签字

2025-2026 学年第 2 学期

实验 7.3 组合求和

1. 实验目的

（1）理解回溯法的求解过程。

（2）分析回溯法的时间复杂度，比较回溯法算法与其他算法的时间效率差异。

（3）学会如何利用回溯法求解具体问题，了解动回溯法的应用范围及在实际应用中的局限性。

2. 实验任务

（1）完成实验 7.3（组合求和问题）的各项要求。

（2）用 C++语言实现该算法并进行测试。

（3）撰写实验报告，实验报告内容包括实验目的、实验任务、实验环境、实验步骤、实验结果和实验总结等部分。

3. 实验环境

3.1 硬件环境

（1） 计算机：Legion R9000P ADR10H

（2） CPU: AMD Ryzen 9 8945HX with Radeon Graphics

（3） RAM：32GB

3.2 软件环境

（1） 操作系统：Windows11 家庭中文版

（2） 开发工具：Visual Studio 2026

4. 实验步骤及结果

4.1 实验预习

4.1.1 写出采用回溯法求解上述问题的目标函数，约束条件以及限界函数

设 list 中的数字为 a_1，a_2，…，a_n。设 x_i 表示第 i 个数字是否被选中，x_i 的取值为 0 或 1。

（1）目标函数

目标函数为：

$$\sum a_i x_i = t$$

即从 list 中选择若干个数字，使它们的和等于目标值 t。

在程序实现中，使用 left 表示当前距离目标值还差的数值。当 left = 0 时，说明当前组合已经构成一个可行解。

（2）约束条件

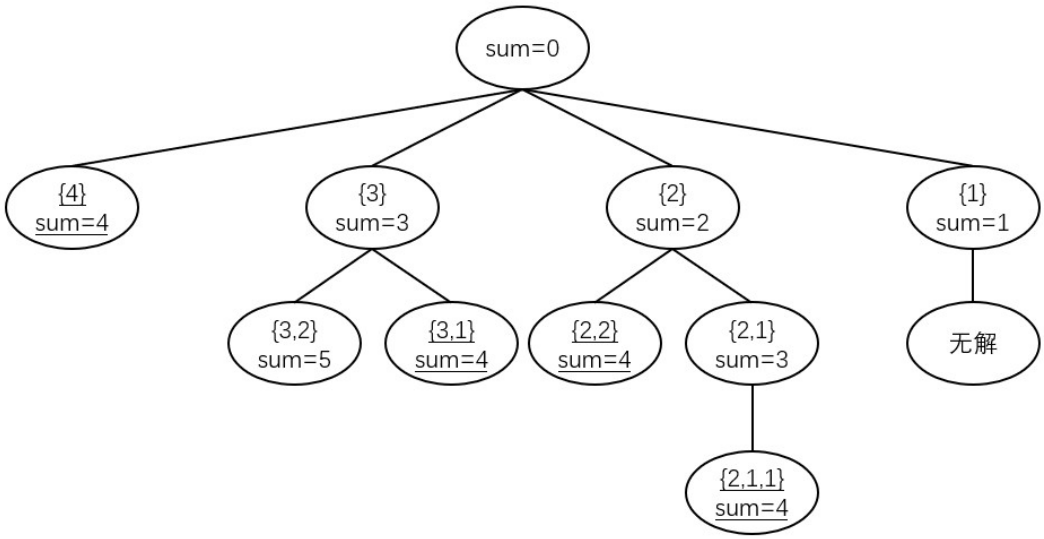
1. 每个数字最多只能使用一次，但如果 list 中有多个相同数字，则可以按照其出现次数分别使用。
2. 每个表达式中的加数必须按降序排列。
3. 输出结果中不能出现相同的加法表达式。
4. 当前组合的元素和不能超过目标值 t。
5. 输入包含多组测试数据，当 n = 0 时输入结束。

（3）限界函数

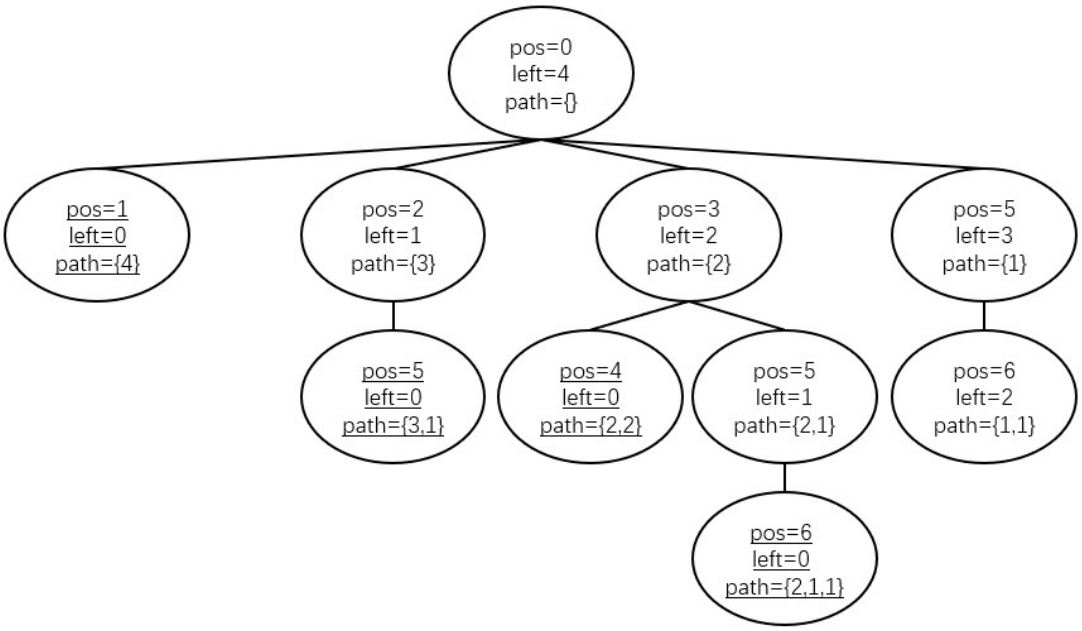
在回溯搜索过程中，为了减少无效搜索，需要设置限界函数进行剪枝。

1. 当 left = 0 时，当前路径为一个可行解，保存该组合。
2. 当当前数字大于 left 时，该数字不能加入当前组合，继续判断后面的数字。
3. 当同一层搜索中出现相同数字时，跳过后面的重复数字，避免产生重复表达式。
4. 当所有数字均已搜索完仍未满足 left = 0 时，当前路径搜索失败。

4.1.2 画出采用回溯法在求解样例输入 1 时的解空间树



4.1.3 画出采用回溯法在求解样例输入 1 时的搜索空间树



4.1.4 用 C/C++语言编写程序求解上述问题

```
#include <bits/stdc++.h>
using namespace std;

int t, n;
vector<int> nums, path;
vector<vector<int>> ans;

void dfs(int pos, int left) {
    if (left == 0) {
        ans.push_back(path);
        return;
    }

    for (int i = pos; i < n; i++) {
        if (i > pos && nums[i] == nums[i - 1]) continue;
        if (nums[i] > left) continue;

        path.push_back(nums[i]);
        dfs(i + 1, left - nums[i]);
        path.pop_back();
```

```
    }
}

int main() {
    while (cin >> t >> n) {
        if (n == 0) break;

        nums.resize(n);
        for (int i = 0; i < n; i++) {
            cin >> nums[i];
        }

        sort(nums.begin(), nums.end(), greater<int>());

        path.clear();
        ans.clear();

        dfs(0, t);

        if (ans.empty()) {
            cout << "NONE" << endl;
        } else {
            for (int i = 0; i < ans.size(); i++) {
                for (int j = 0; j < ans[i].size(); j++) {
                    if (j > 0) cout << "+";
                    cout << ans[i][j];
                }
                cout << endl;
            }
        }
    }

    return 0;
}
```

4.1.5 分析上述算法的时间复杂度，给出复杂度结果

设每组测试数据中有 n 个数字。程序首先对输入数组进行排序，排序的时间复杂度为 O(nlogn)。

在回溯搜索过程中，每个数字都有选取和不选取两种可能，因此在最坏情况下需要搜索所有子集，搜索规模为 2^n。虽然程序中加入了剪枝和去重操作，可以减少部分无效搜索，但在最坏情况下，回溯搜索的时间复杂度仍为 $O(2^n)$。

因此，每组测试数据的总时间复杂度为：$O(nlogn + 2^n)$

由于 2^n 在数据规模增大时占主导地位，所以算法的时间复杂度通常记为：$O(2^n)$

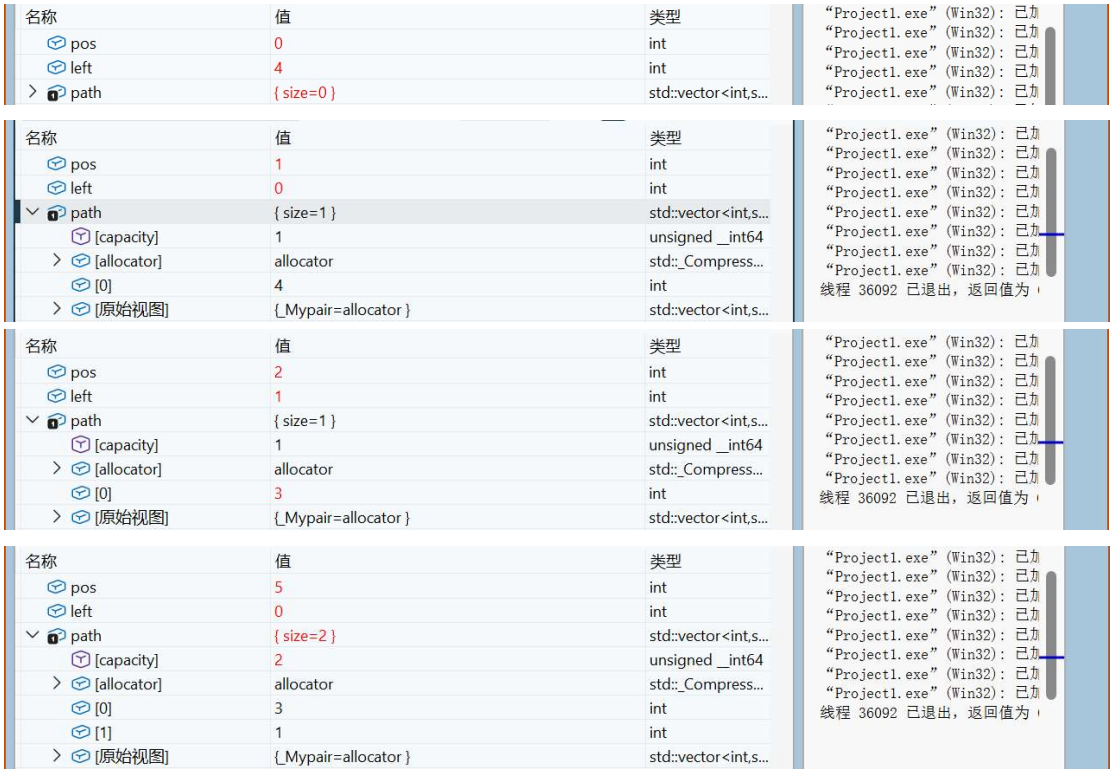
4.2 上机实验

4.2.1 上机验证程序在样例输入 1 时的执行过程是否与步骤（3）所得到的搜索空间树一致，写出程序调试过程中搜索空间树上每个结点的值，并与前面的结果进行比对

测试用例 1 的程序运行结果：



程序单步调试过程中每个结点的变量值：





从单步调试截图可以看出，程序进入 dfs 函数时各结点的 pos、left 和 path 变量值与搜索空间树中的结点编号一致。程序依次得到 4、3+1、2+2 和 2+1+1 四个可行解，因此程序执行过程与前面绘制的搜索空间树一致。

4.2.2 设计新的测试用例，对程序进行进一步验证

测试输入：

8 7 10 1 2 7 6 1 5

手工计算结果：

根据题意，每个数字最多只能使用其在 list 中出现的次数，并且输出表达式中

的加数必须按降序排列。

能够组成目标值 8 的组合有：

7+1

6+2

6+1+1

5+2+1

程序运行结果：



程序输出结果与手工计算结果一致。

5. 实验总结

本次实验使用回溯法求解组合求和问题。实验中，先对输入的数字集合按降序排序，使程序输出的加法表达式满足降序要求。然后从当前位置开始依次选择数字，并用 left 表示当前距离目标值还差的数值。当 left 等于 0 时，说明当前路径中的数字之和正好等于目标值，此时将当前路径保存为一个可行解。

在搜索过程中，程序通过两个主要方式减少无效搜索。第一，当前数字大于 left 时，说明该数字不能加入当前组合，因此跳过该数字。第二，当同一层中出现相同数字时，程序跳过后面的重复数字，从而避免产生重复表达式。

通过样例输入和自定义测试用例的验证，程序能够正确输出所有满足条件的组合，并能在无解时输出 NONE。实验结果说明，回溯法适合用于求解组合类问题，其优点是思路清晰、实现直接；缺点是最坏情况下仍然需要搜索大量状态，时间复杂度较高。通过本次实验，我进一步理解了回溯法的搜索过程、剪枝思想和重复组合的处理方法。